

Documentation de gestion de projet — Zoo d'Arcadia

1. Méthodologie de gestion de projet

Ce projet a été mené seule, dans le cadre d'une formation à distance chez Studi, sans équipe ni stage en entreprise associé. Les méthodes agiles classiques (mêlées quotidiennes, planification de sprint en équipe) ne s'appliquaient donc pas telles quelles : la gestion de projet s'est plutôt organisée autour d'un suivi personnel des tâches, proche d'un kanban individuel, complété par l'organisation Git elle-même. Le kanban utilisé est un tableau Trello : <https://trello.com/b/deKcxMx8> (voir aussi la section 4).

Le découpage du travail en branches Git - une branche par fonctionnalité, fusionnée dans `develop` une fois terminée et testée, puis `develop` fusionnée périodiquement dans `main` - a fonctionné comme un second niveau de suivi d'avancement : l'état d'une fonctionnalité (en cours, terminée, intégrée) se lisait directement dans l'état des branches, en complément du kanban.

Cette organisation n'a toutefois pas été en place dès le premier jour. La partie front-end de 2025 ne compte que quelques commits directs, et le passage du site statique à l'application PHP a lui aussi été enregistré en un seul commit sur `develop`, le 29 août 2026. Je n'avais tout simplement pas encore suivi les cours sur Git à ce moment-là et ai implémenté ce système de travail dès que j'ai pu. Le flux par branches a été adopté dès le lendemain, et les huit fonctionnalités suivantes le respectent strictement : chacune a sa branche, sa fusion en `--no-ff` vers `develop`, puis son passage en `main`.

2. Déroulement réel du projet

Le projet s'est déroulé en trois phases, dont les deux dernières sont datées avec précision par l'historique du dépôt.

La **partie front-end** a été développée en majorité courant 2024/2025 et rendue une première fois en septembre 2025, sous forme d'un dossier ne couvrant que cette partie ; les commits correspondants vont du 15 au 22 septembre 2025. Le travail antérieur n'est pas versionné : je n'avais pas encore appris à utiliser Git à ce stade de la formation. J'ai ensuite apporté quelques modifications lors du passage en PHP du site et du développement back-end pour avoir un code plus fluide et simple à maintenir.

La **partie back-end** a été développée majoritairement entre août et septembre 2026, en réutilisant l'ensemble des maquettes et du cahier des charges déjà validés pour construire les espaces professionnels par-dessus le front-end existant.

Le **déploiement** a occupé les 6 et 7 septembre 2026 : mise en ligne chez alwaysdata, création de la base distante, compilation de l'extension MongoDB, branchement d'un cluster MongoDB Atlas, puis tests en production. La démarche complète est détaillée dans la documentation technique.

La flexibilité offerte par la plateforme Studi sur le planning de passage des examens a été déterminante dans l'organisation de ce projet : plusieurs phases de difficultés personnelles et financières ont empêché de suivre les cours et d'avancer sur le projet de façon continue, ce qui a nécessité, à chaque reprise, un temps de remise en route pour retrouver le fil de ce qui avait déjà été appris. Le planning n'a donc pas été construit autour de sprints à échéance fixe, mais autour d'une progression fonctionnalité par fonctionnalité, reprise dès que le temps disponible le permettait.

3. Environnement humain et technique

Le projet a été réalisé en autonomie, sans stage en développement web associé à la période de formation. L'environnement technique repose sur XAMPP (Apache, PHP, MySQL) en local, MongoDB avec MongoDB Compass pour la base NoSQL, VS Code comme éditeur (avec les extensions Prettier, Live Server et Quokka), et Git/GitHub pour le versionnement. Pour la partie back-end, un accompagnement complémentaire avec Claude Code a permis de progresser plus efficacement sur le temps disponible, en laissant les tâches les plus répétitives à l'assistant après une prise en main guidée des tâches variées - une méthode déjà détaillée dans l'introduction du dossier.

4. Suivi des tâches (kanban partagé)

Le suivi des tâches est tenu sur un tableau Trello partagé publiquement : <https://trello.com/b/deKcxMx8>

Conformément à l'énoncé, il comporte cinq colonnes :

Colonne	Contenu
Backlog	toutes les fonctionnalités prévues, ordonnées par priorité
À faire	ce qui est prévu pour la prochaine étape de développement
En cours	ce sur quoi le travail porte actuellement

Colonne	Contenu
Terminé (develop)	fusionné dans la branche de développement
Fusionné (main)	passé en branche principale

Chaque carte porte le nom de la branche Git correspondante. Il permet de vérifier dans le graphe du dépôt que la carte « US 4 », par exemple, correspond bien à la branche `feature/page-habitats`.

La colonne **Backlog** ne contient pas des fonctionnalités oubliées mais les évolutions identifiées et volontairement écartées, avec leur priorité : la protection CSRF par jeton en priorité haute, le téléversement d'images en priorité moyenne, la gestion de plusieurs images par habitat et l'envoi réel de courriels en priorité basse. Ces mêmes points figurent comme limites assumées dans le dossier de projet.

Ce tableau a été ouvert dès la phase de conception front-end, en 2025, avec un découpage par pages à réaliser. Il a été restructuré pour la phase back-end afin d'adopter les cinq colonnes demandées et de refléter le flux Git par branches. Les cartes des premières fonctionnalités ont donc été reclassées a posteriori ; leur ordre correspond au déroulement réel, recoupable avec l'historique du dépôt.

5. Limites de gestion assumées

Le projet ayant été mené seule et avec un planning très irrégulier, certains arbitrages ont porté directement sur le temps disponible plutôt que sur des choix techniques : c'est le cas de la protection CSRF, du téléversement sécurisé des images, ou de l'envoi réel de courriels, déjà détaillés comme limites assumées dans le dossier principal.